**Programare Web / Dezvoltarea Aplicațiilor Web**
Tehnologia Informației, anul IV · Informatică, anul II · Anul universitar 2026–2027, semestrul I

LABORATORUL 01

# Mediul de lucru și template-ul de proiect

*Instalăm instrumentele, clonăm template-ul, ne creăm branch-ul personal și pornim pentru prima dată aplicația completă: backend, frontend și baza de date.*

| | |
|---|---|
| **Tema** | Săptămâna 1 din planul disciplinei (examene.pe.ub.ro): introducere, mediul de lucru și instrumentele |
| **Durata** | 2 ore (instalările mai lungi se pot termina acasă) |
| **Aveți nevoie de** | laptopul propriu (Windows, macOS sau Linux) cu drepturi de administrator, conexiune la internet și o adresă de e-mail |
| **La final aveți** | template-ul clonat și pornit, branch-ul personal `nume.prenume` pe GitHub și primul vostru commit |

## 1. Obiective

La finalul laboratorului veți putea:

- explica pe scurt ce sunt backend-ul, frontend-ul, un API, Docker și Git;
- instala și verifica instrumentele pe care le folosim tot semestrul;
- clona template-ul de proiect și lucra pe branch-ul vostru personal;
- porni aplicația în containere Docker și verifica, în browser, că totul funcționează;
- face o modificare, un commit și un push, apoi vedea commit-ul pe GitHub.

## 2. Pe scurt: cum arată o aplicație web modernă

O aplicație web are două părți care comunică prin rețea folosind protocolul HTTP [1], [2]. **Backend-ul** rulează pe server, lucrează cu baza de date și răspunde la cereri. **Frontend-ul** rulează în browserul utilizatorului și afișează interfața. Cele două vorbesc printr-un **API**: o listă de adrese (de exemplu `/api/products`) la care frontend-ul trimite cereri și primește date în format JSON.

| Noțiune | Ce este | În template-ul nostru |
|---|---|---|
| Backend | Programul de pe server care primește cererile, aplică regulile aplicației și citește sau scrie în baza de date. | PHP 8.5 + Symfony 8.1, folderul `backend/` |
| Frontend | Interfața din browser: pagini, butoane, formulare. | React 19 + Vite, folderul `frontend/` |
| API | Adresele prin care frontend-ul cere și trimite date; răspunsurile sunt în JSON. | `/api/health`, `/api/categories`, `/api/products` |
| Baza de date | Locul în care datele rămân salvate. | PostgreSQL 18, accesată prin Doctrine ORM |
| Docker | Rulează fiecare serviciu într-un **container** izolat, la fel pe orice calculator [5]. | `daw-nginx`, `daw-php`, `daw-postgres`, `daw-react` |
| Git și GitHub | Git păstrează istoricul modificărilor; GitHub găzduiește proiectul online [4]. | repository-ul cadrului didactic, branch-ul vostru `nume.prenume` |

> **Cum lucrăm.** Laboratoarele urmează planul săptămânal al disciplinei, publicat de dl Pruteanu pe examene.pe.ub.ro. Detaliile de instalare și soluțiile la problemele frecvente sunt în documentul *Metodologie de lucru și template de proiect* [6].

# 3. Instalarea instrumentelor

## 3.1 Contul GitHub

1. Creați un cont pe github.com cu username-ul de forma `prenume-nume` (de exemplu `ion-popescu`), ca să vă recunoaștem ușor în istoricul proiectului.
2. ★ Ca studenți, puteți cere gratuit pachetul GitHub Student Developer Pack, care include și licențe JetBrains.

## 3.2 Programele

| Program | La ce îl folosim | Descărcare |
|---|---|---|
| Docker Desktop | pornește containerele aplicației | docker.com |
| Git | istoricul codului; pe Windows include **Git Bash** | git-scm.com |
| Editor | VS Code sau PhpStorm/WebStorm (gratuite pentru studenți) | code.visualstudio.com · jetbrains.com |
| DBeaver | vedem tabelele din baza de date | dbeaver.io |
| Postman | trimitem cereri către API | postman.com |

> **Atenție.** Pe Windows, Docker Desktop are nevoie de virtualizare activată (WSL 2 sau Hyper-V). Dacă la pornire primiți o eroare despre virtualizare, urmați pașii din metodologie [6].

## 3.3 Verificarea instalărilor

Porniți Docker Desktop și așteptați să indice că rulează. Deschideți un terminal (pe Windows, **Git Bash**) și scrieți, pe rând:

```
docker --version
docker compose version
git --version
```

> **Rezultat.** Fiecare comandă afișează un număr de versiune, de exemplu `Docker version 28...` sau `git version 2...`. Dacă primiți „command not found”, programul nu este instalat corect sau terminalul a fost deschis înainte de instalare: închideți-l și deschideți-l din nou.

# 4. Git: configurare, clonare și branch-ul personal

## 4.1 Configurarea Git (o singură dată pe calculator)

> **De ce.** Fiecare commit poartă un nume și un e-mail. Folosiți aceeași adresă ca la GitHub, ca să vi se atribuie contribuțiile.

```
git config --global user.name "Ion Popescu"
git config --global user.email "adresa.voastra@exemplu.ro"
git config --global pull.rebase false
```

## 4.2 Clonarea template-ului

### 4.2.a Creați un folder pentru proiecte

Alegeți un loc simplu pe disc, fără spații și fără diacritice în cale, care nu este pe Desktop și nici într-un folder sincronizat (OneDrive, iCloud). Recomandăm un folder numit `proiecte`.

- **Cu mouse-ul, pe Windows:** deschideți **File Explorer**, intrați în `C:\` (sau în **Documente**), click dreapta pe un spațiu liber → **Nou → Folder** (*New → Folder*) și numiți-l `proiecte`.
- **Cu mouse-ul, pe macOS:** deschideți **Finder**, intrați în folderul personal (cel cu numele vostru), click dreapta → **New Folder** și numiți-l `proiecte`.
- **Din terminal:** pe Windows, în Git Bash: `mkdir -p /c/proiecte`; pe macOS: `mkdir -p ~/proiecte`. Opțiunea `-p` nu dă eroare dacă folderul există deja.

### 4.2.b Deschideți un terminal în folderul proiecte

- **Windows:** în File Explorer intrați în folderul `proiecte`, click dreapta pe un spațiu liber → **Open Git Bash here** (pe Windows 11, mai întâi **Show more options**).
- **macOS:** în Finder, click dreapta pe folderul `proiecte` → **Services → New Terminal at Folder**. Dacă opțiunea lipsește, o activați din **System Settings → Keyboard → Keyboard Shortcuts → Services → Files and Folders**, sau deschideți aplicația **Terminal** și scrieți `cd ~/proiecte`.

Verificați locul în care vă aflați cu `pwd`: calea afișată trebuie să se termine cu `/proiecte`.

### 4.2.c Clonați proiectul

```
git clone https://github.com/cezarpapara/daw-template-ti-info-2026-2027.git
```

**De ce.** „A clona” înseamnă a descărca proiectul de pe GitHub pe calculator, împreună cu tot istoricul lui. Comanda creează în folderul curent un folder nou, `daw-template-ti-info-2026-2027`, cu tot conținutul proiectului. Clonarea se face **o singură dată**; dacă o repetați în același loc, primiți eroarea `destination path already exists`.

### 4.2.d Verificați că proiectul s-a copiat complet

Deschideți folderul `proiecte/daw-template-ti-info-2026-2027` în File Explorer sau Finder și comparați-l cu pagina proiectului de pe GitHub (github.com/cezarpapara/daw-template-ti-info-2026-2027): trebuie să vedeți aceleași fișiere și foldere, adică `README.md`, `backend`, `frontend` și `postman`. Din terminal, același lucru: `cd daw-template-ti-info-2026-2027`, apoi `ls`.

**Dacă apare o eroare.** Dacă la clonare sau la verificare apare un mesaj de eroare, nu insistați la întâmplare: faceți o captură de ecran și anunțați cadrul didactic, care vă va îndruma.

### 4.2.e Deschideți proiectul în editor

Deschideți în editor **exact** folderul `daw-template-ti-info-2026-2027`: nu folderul `proiecte` (un nivel mai sus) și nici `backend` sau `frontend` (un nivel mai jos). Calea arată, de exemplu, așa:

```
Windows:  C:\proiecte\daw-template-ti-info-2026-2027
macOS:    /Users/numele-vostru/proiecte/daw-template-ti-info-2026-2027
```

- **Visual Studio Code** (gratuit): **File → Open Folder…**, alegeți folderul și confirmați **Trust the authors** dacă vi se cere.
- **PhpStorm** sau **WebStorm** (gratuite pentru studenți): **File → Open…**, selectați folderul și apăsați **Open**, apoi **Trust Project**.

### 4.2.f Terminalul din editor

De aici înainte folosim terminalul integrat în editor:

- **VS Code:** meniul **Terminal → New Terminal** (sau **Ctrl + `**). Pe Windows, din săgeata de lângă **+** alegeți **Git Bash**.
- **PhpStorm / WebStorm:** meniul **View → Tool Windows → Terminal** (sau **Alt + F12**, pe macOS **⌥ F12**), ori tab-ul **Terminal** din partea de jos a ferestrei.

> **Avantajul terminalului din editor.** Se deschide direct în **rădăcina proiectului** (*root*), adică exact în folderul din care se rulează toate comenzile din acest îndrumar. Cât timp nu vă mutați cu `cd` în alt folder, sunteți în locul potrivit; verificați oricând cu `pwd`. Dacă v-ați mutat, reveniți cu `cd` sau deschideți un terminal nou.

## 4.3 Branch-ul personal

### 4.3.a Creați branch-ul personal

Branch-ul `main` este copia principală a proiectului, pe care lucrează doar cadrul didactic. Fiecare student lucrează pe branch-ul lui, numit `nume.prenume` (litere mici, fără diacritice, cu punct între nume și prenume). Înlocuiți `popescu.ion` cu numele vostru:

```
git checkout -b popescu.ion        # sau, echivalent: git switch -c popescu.ion
```

`git checkout -b` **creează** branch-ul și vă **mută** pe el. Comanda se rulează o singură dată; a doua oară primiți eroarea `already exists`.

### 4.3.b Primul push: legarea branch-ului de GitHub

```
git push -u origin popescu.ion
```

- `origin` este numele sub care Git ține minte adresa proiectului de pe GitHub;
- `popescu.ion` este branch-ul pe care îl trimiteți;
- `-u` (prescurtarea lui `--set-upstream`) **leagă o singură dată** branch-ul local de cel de pe GitHub. După acest prim push, comenzile `git push` și `git pull`, scrise fără alți parametri, știu singure unde să trimită și de unde să aducă modificările.

> **Forma completă și forma scurtă.** Mai departe puteți folosi oricând forma completă, `git push origin popescu.ion`, sau forma scurtă, `git push`. Forma scurtă funcționează **numai** dacă ați făcut anterior, o dată, `git push -u origin popescu.ion`; altfel Git răspunde cu `has no upstream branch`.

> **Rezultat.** La primul push se deschide autentificarea GitHub în browser. `git branch` afișează `* popescu.ion`, iar pe GitHub branch-ul vostru apare în lista de branch-uri. Dacă primiți eroarea **403** (*Permission denied*), trimiteți cadrului didactic username-ul de GitHub; după ce acceptați invitația primită pe e-mail, repetați `git push -u origin popescu.ion`.

# 5. Prima pornire a aplicației

Docker Desktop trebuie să fie pornit. Rulați comenzile **una câte una**, în terminalul din editor, așteptând să se termine fiecare:

| Comanda | Ce face | Când se rulează din nou |
|---|---|---|
| `docker compose -f backend/compose.yaml up -d --build` | construiește imaginile și pornește containerele backend-ului (Nginx, PHP, PostgreSQL); prima dată durează câteva minute | fără `--build` la fiecare pornire; cu `--build` doar dacă se modifică fișierele din `backend/docker` |
| `docker exec -it daw-php composer install` | instalează bibliotecile PHP ale proiectului (Symfony, Doctrine etc.) în `backend/app/vendor` | doar prima dată și când `composer.json` se schimbă (de exemplu după preluarea noutăților de pe `main`) |
| `docker exec -it daw-php php bin/console doctrine:database:create --if-not-exists` | creează baza de date `daw` | o singură dată (repetată nu strică nimic, dar nu mai este necesară) |

| Comanda | Ce face | Când se rulează din nou |
|---|---|---|
| `docker exec -it daw-php php bin/console doctrine:migrations:migrate` | creează tabelele și datele de exemplu; la întrebare răspundeți `yes` | doar când apar migrări noi (ale voastre sau de pe `main`); fără migrări noi afișează *Already at the latest version* |
| `docker compose -f frontend/compose.yaml up -d --build` | construiește și pornește containerul React (instalează pachetele JavaScript în container) | fără `--build` la fiecare pornire; cu `--build` doar dacă se modifică fișierele din `frontend/docker` |

> **Atenție.** Nu repetați clonarea și nici crearea branch-ului: ele se fac o singură dată. La orele următoare porniți proiectul după pașii din „Începutul fiecărui laborator”.

> **Rezultat.** localhost:8080/api/health afișează „Salut din Symfony!”, localhost:8080/api/doc afișează documentația API (Swagger), iar localhost:5173 deschide aplicația, cu o casetă verde pe pagina Acasă.

> **Sfat.** Comanda `docker ps` listează containerele pornite. Trebuie să apară `daw-nginx`, `daw-php`, `daw-postgres` și `daw-react`. Dacă unul lipsește, consultați secțiunea de probleme frecvente din metodologie [6].

## 6. Exerciții

1. **Prima modificare.** Deschideți `frontend/app/src/pages/HomePage.jsx` (pagina pentru ruta `/`). Găsiți un text vizibil, de exemplu titlul din `<h1>`, și adăugați la el numele vostru. Salvați și verificați pe http://localhost:5173 (dacă pagina nu se actualizează singură, apăsați **F5**).
2. **Commit și push.** Salvați modificarea în istoric și trimiteți-o pe GitHub:

   ```
   git status
   git add .
   git commit -m "Laboratorul 01: numele meu pe pagina Acasă"
   git push origin popescu.ion     # sau doar: git push (dupa git push -u)
   ```

3. Pe GitHub, deschideți repository-ul, alegeți branch-ul vostru din lista **main ▾** și apăsați **Commits**. Commit-ul vostru trebuie să apară primul.
4. ★ (opțional) În DBeaver creați o conexiune PostgreSQL: host `localhost`, port `5432`, baza `daw`, utilizator `daw`, parolă `daw`. Găsiți tabelele categoriilor și produselor.
5. ★ (opțional) În Postman: **Import** → fișierul `postman/DAW-API.postman_collection.json`. Trimiteți cererea `GET /api/products`. Apoi, din Swagger (**Try it out**), adăugați o categorie cu `POST /api/categories`, folosind exemplul de corp propus, și verificați-o în DBeaver.

## 7. Oprirea containerelor

La sfârșitul lucrului, opriți containerele (datele din baza de date rămân):

```
docker compose -f backend/compose.yaml stop
docker compose -f frontend/compose.yaml stop
```

Data viitoare urmați pașii de la începutul fiecărui laborator:

1. Porniți **Docker Desktop** și așteptați indicația *Engine running*.
2. Deschideți proiectul în editor și terminalul integrat (sunteți automat în rădăcina proiectului).
3. Verificați pe ce branch sunteți:

   ```
   git branch          # ramura curenta are * in fata
   git status          # primul rand: On branch ...
   ```

4. Doar dacă **nu** sunteți deja pe branch-ul vostru, treceți pe el (dacă sunteți, săriți acest pas):

```
git checkout popescu.ion        # sau: git switch popescu.ion
```

5. Porniți containerele (fără `--build`):

```
docker compose -f backend/compose.yaml up -d
docker compose -f frontend/compose.yaml up -d
```

## 8. Ce arătați la final

- rezultatul celor trei comenzi de verificare (Docker, Docker Compose, Git);
- paginile `/api/health`, `/api/doc` și aplicația React cu caseta verde;
- commit-ul de la exercițiul 2, vizibil pe GitHub, pe branch-ul vostru `nume.prenume`.

## Resurse pentru exersare

- **W3Schools – Git (cu exerciții):** https://www.w3schools.com/git/
- **Git, în limba română (Marius Ciurea, video):** https://www.youtube.com/watch?v=iVpuVkVeBcw
- **Pro Git, traducere în română:** https://git-scm.com/book/ro/v2
- **Docker pentru începători (Mosh, video în engleză):** https://www.youtube.com/watch?v=pTFZFxd4hOI
- **Docker – documentație oficială:** https://docs.docker.com/
- **Planul disciplinei (dl Pruteanu):** http://examene.pe.ub.ro/

## Bibliografie

**[1]** E. Pruteanu și I. Furdu, *Modele, tehnologii, servicii și aplicații integrate de tip R.I.A. pentru web*. Bacău: Alma Mater, 2022.

**[2]** S. Buraga, *Tehnologii Web*. București: Matrix Rom, 2001.

**[3]** R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*, teză de doctorat, University of California, Irvine, 2000.

**[4]** S. Chacon și B. Straub, *Pro Git*, ed. a 2-a. Apress, 2014. Traducere în română: git-scm.com/book/ro/v2

**[5]** Docker Inc., *Docker Docs*. docs.docker.com

**[6]** C.-M. Papară, *Metodologie de lucru și template de proiect – Programare Web / Dezvoltarea Aplicațiilor Web*, UVAB, 2026.